

Asynchronous Patterns in Node.js

Introduction

In Node.js, asynchronous programming is the foundation of handling multiple operations without blocking the execution thread. Unlike traditional synchronous code where each operation waits for the previous one to complete, asynchronous patterns allow your program to continue executing while waiting for time-consuming operations like file reading, database queries, or API calls.

Think of it like a restaurant kitchen. In a synchronous kitchen, one chef prepares the entire meal from start to finish before starting the next order. In an asynchronous kitchen, multiple chefs work on different parts of different orders simultaneously, maximizing efficiency.

Node.js provides several patterns to handle asynchronous operations, each evolving to solve problems of its predecessor. This document covers four fundamental patterns: Callbacks, Promises, Async/Await, and EventEmitter.

1. Callbacks: The Foundation of Asynchronous Programming

What are Callbacks?

A callback is a function passed as an argument to another function, which is executed after the completion of an operation. In Node.js, callbacks are the traditional way to handle asynchronous operations.

The Error-First Callback Convention

Node.js follows a standard pattern called "error-first callbacks" where the first parameter of the callback function is reserved for an error object, and subsequent parameters contain the result data. This convention ensures consistent error handling across Node.js applications.

Basic Callback Structure

```
// Basic callback demonstration
function fetchUserData(userId, callback) {
  // Simulate async operation with setTimeout
  setTimeout(() => {
    // Simulating data retrieval
    const user = { id: userId, name: 'Alice', email: 'alice@example.com' };
    callback(null, user); // null = no error, user = data
  }, 1000);
}

// Using the callback
console.log('Fetching user data...');
fetchUserData(101, (error, user) => {
  if (error) {
    console.error('Error:', error.message);
    return;
  }
  console.log('User data received:', user);
});
console.log('Request sent, waiting for response...');
```

Output:

```
Fetching user data...
Request sent, waiting for response...
(after 1 second)
User data received: { id: 101, name: 'Alice', email: 'alice@example.com' }
```

Key Observation: Notice how "Request sent, waiting for response..." prints before the user data. This demonstrates non-blocking behavior.

Error Handling in Callbacks

```
// Function with potential error
function readUserFile(filename, callback) {
  setTimeout(() => {
    // Simulate file not found error
```

```
    if (filename !== 'users.txt') {
      const error = new Error(`File not found: ${filename}`);
      callback(error, null);
      return;
    }

    // Success case
    const data = 'User1, User2, User3';
    callback(null, data);
  }, 500);
}

// Handling both error and success
console.log('Reading file...');
readUserFile('config.txt', (error, data) => {
  if (error) {
    console.error('Failed to read file:', error.message);
    return;
  }
  console.log('File content:', data);
});

// Correct usage
readUserFile('users.txt', (error, data) => {
  if (error) {
    console.error('Failed to read file:', error.message);
    return;
  }
  console.log('File content:', data);
});
```

Output:

```
Reading file...
Failed to read file: File not found: config.txt
File content: User1, User2, User3
```

The Callback Hell Problem

When you need to perform multiple asynchronous operations in sequence, callbacks can lead to deeply nested code, commonly called "callback hell" or "pyramid of doom".

```
// Demonstrating callback hell
function getUser(userId, callback) {
  setTimeout(() => {
    callback(null, { id: userId, name: 'Alice' });
  }, 500);
}

function getUserOrders(userId, callback) {
  setTimeout(() => {
    callback(null, ['Order1', 'Order2']);
  }, 500);
}

function getOrderDetails(orderId, callback) {
  setTimeout(() => {
    callback(null, { orderId, amount: 100, status: 'delivered' });
  }, 500);
}

// Nested callbacks - difficult to read and maintain
getUser(1, (error, user) => {
  if (error) {
    console.error('Error fetching user:', error);
    return;
  }
  console.log('User:', user.name);

  getUserOrders(user.id, (error, orders) => {
    if (error) {
      console.error('Error fetching orders:', error);
      return;
    }
    console.log('Orders:', orders);

    getOrderDetails(orders[0], (error, details) => {
      if (error) {
        console.error('Error fetching order details:', error);
        return;
      }
      console.log('Order details:', details);
    });
  });
});
```

Output:

```
User: Alice
Orders: ['Order1', 'Order2']
Order details: { orderId: 'Order1', amount: 100, status: 'delivered' }
```

Why This is Problematic:

- Code becomes difficult to read and maintain
- Error handling is repetitive
- Indentation grows with each nested operation
- Hard to debug and test

2. Promises: A Better Way to Handle Async Operations

What are Promises?

A Promise is an object representing the eventual completion or failure of an asynchronous operation. It's a cleaner alternative to callbacks, introduced natively in ES6 (ECMAScript 2015). Think of a promise as a receipt you get when ordering food at a restaurant—it's not the food itself, but a guarantee that the food will arrive (or you'll be notified if there's a problem).

Promise States

A Promise can be in one of three states:

- **Pending:** Initial state, neither fulfilled nor rejected
- **Fulfilled:** Operation completed successfully
- **Rejected:** Operation failed

Creating a Basic Promise

```
// Creating a simple promise
function fetchUserDataPromise(userId) {
  return new Promise((resolve, reject) => {
    setTimeout(() => {
      if (userId <= 0) {
        reject(new Error('Invalid user ID'));
        return;
      }

      const user = { id: userId, name: 'Bob', email: 'bob@example.com' };
      resolve(user);
    }, 1000);
  });
}

// Using the promise
console.log('Fetching user with promise...');
fetchUserDataPromise(102)
  .then(user => {
    console.log('User received:', user);
  })
  .catch(error => {
    console.error('Error:', error.message);
  });

console.log('Request initiated...');
```

Output:

```
Fetching user with promise...
Request initiated...
(after 1 second)
User received: { id: 102, name: 'Bob', email: 'bob@example.com' }
```

Promise Chaining: Solving Callback Hell

Promises can be chained, allowing sequential asynchronous operations to be written in a flat, readable manner.

```
// Converting previous callback example to promises
function getUserPromise(userId) {
  return new Promise((resolve, reject) => {
    setTimeout(() => {
      if (userId <= 0) {
        reject(new Error('Invalid user ID'));
        return;
      }
      resolve({ id: userId, name: 'Alice' });
    }, 500);
  });
}

function getUserOrdersPromise(userId) {
  return new Promise((resolve, reject) => {
    setTimeout(() => {
      resolve(['Order1', 'Order2']);
    }, 500);
  });
}

function getOrderDetailsPromise(orderId) {
  return new Promise((resolve, reject) => {
    setTimeout(() => {
      resolve({ orderId, amount: 100, status: 'delivered' });
    }, 500);
  });
}

// Flat promise chain - much more readable
getUserPromise(1)
  .then(user => {
    console.log('User:', user.name);
    return getUserOrdersPromise(user.id);
  })
  .then(orders => {
    console.log('Orders:', orders);
    return getOrderDetailsPromise(orders[0]);
  })
  .then(details => {
    console.log('Order details:', details);
  })
  .catch(error => {
    console.error('Error in chain:', error.message);
  });
```

Output:

```
User: Alice
Orders: ['Order1', 'Order2']
Order details: { orderId: 'Order1', amount: 100, status: 'delivered' }
```

Benefits of Promise Chaining:

- Flat code structure instead of nested callbacks
- Single `catch` block can handle all errors in the chain
- Easy to read and understand the flow
- Each `then` returns a new promise, enabling chaining

The `finally()` Method

The `finally()` method is executed regardless of whether the promise is fulfilled or rejected. It's useful for cleanup operations.

```
// Using finally for cleanup
function fetchDataWithLoading(userId) {
  console.log('Loading: true');

  return new Promise((resolve, reject) => {
    setTimeout(() => {
      if (userId === 999) {
        reject(new Error('Server error'));
        return;
      }
      resolve({ id: userId, name: 'Charlie' });
    }, 1000);
  });
}

// Success case
fetchDataWithLoading(103)
  .then(data => {
    console.log('Data received:', data);
  })
  .catch(error => {
```



```
        console.error('Error occurred:', error.message);
    })
    .finally(() => {
        console.log('Loading: false');
        console.log('Cleanup completed');
    });

// Error case
console.log('\n--- Error scenario ---');
fetchDataWithLoading(999)
    .then(data => {
        console.log('Data received:', data);
    })
    .catch(error => {
        console.error('Error occurred:', error.message);
    })
    .finally(() => {
        console.log('Loading: false');
        console.log('Cleanup completed');
    });
```

Output:

```
Loading: true
--- Error scenario ---
Loading: true
(after 1 second)
Data received: { id: 103, name: 'Charlie' }
Loading: false
Cleanup completed
Error occurred: Server error
Loading: false
Cleanup completed
```

Multiple Promises: Promise.all() and Promise.race()

```
// Promise.all - waits for all promises to resolve
function fetchUser() {
    return new Promise(resolve => {
```

```
        setTimeout(() => resolve({ name: 'Alice' }), 500);
    });
}

function fetchPosts() {
    return new Promise(resolve => {
        setTimeout(() => resolve(['Post1', 'Post2']), 700);
    });
}

function fetchComments() {
    return new Promise(resolve => {
        setTimeout(() => resolve(['Comment1', 'Comment2']), 300);
    });
}

console.log('Fetching all data...');
Promise.all([fetchUser(), fetchPosts(), fetchComments()])
    .then(([user, posts, comments]) => {
        console.log('User:', user);
        console.log('Posts:', posts);
        console.log('Comments:', comments);
        console.log('All data loaded successfully');
    })
    .catch(error => {
        console.error('One or more requests failed:', error);
    });

// Promise.race - returns first resolved/rejected promise
console.log('\n--- Race scenario ---');
Promise.race([fetchUser(), fetchPosts(), fetchComments()])
    .then(result => {
        console.log('First result:', result);
    });
});
```

Output:

```
Fetching all data...
```

```
--- Race scenario ---
```

```
(after 300ms)
```

```
First result: ['Comment1', 'Comment2']
```

```
(after 700ms - when all complete)
```

```
User: { name: 'Alice' }
```

```
Posts: ['Post1', 'Post2']
Comments: ['Comment1', 'Comment2']
All data loaded successfully
```

3. Async/Await: Writing Asynchronous Code Like Synchronous

What is Async/Await?

Async/await is syntactic sugar built on top of Promises, introduced in ES2017 (ES8). It allows you to write asynchronous code that looks and behaves like synchronous code, making it even more readable and maintainable than promise chains.

The `async` keyword is used to declare an async function, and the `await` keyword is used to wait for a Promise to resolve.

Basic Async/Await Usage

```
// Converting promise example to async/await
function fetchUserData(userId) {
  return new Promise((resolve, reject) => {
    setTimeout(() => {
      if (userId <= 0) {
        reject(new Error('Invalid user ID'));
        return;
      }
      resolve({ id: userId, name: 'David', email: 'david@example.com' });
    }, 1000);
  });
}

// Using async/await
async function displayUser() {
  console.log('Fetching user...');
  const user = await fetchUserData(104);
  console.log('User received:', user);
  console.log('Process complete');
}
```

```
displayUser();  
console.log('Function called (runs immediately)');
```

Output:

```
Fetching user...  
Function called (runs immediately)  
(after 1 second)  
User received: { id: 104, name: 'David', email: 'david@example.com' }  
Process complete
```

Key Points:

- `async` function always returns a Promise
- `await` pauses execution until the Promise resolves
- `await` can only be used inside `async` functions
- Code after `await` executes only after the Promise resolves

Error Handling with Try-Catch

With `async/await`, error handling becomes more intuitive using traditional try-catch blocks, similar to synchronous code.

```
// Error handling with async/await  
async function getUserWithErrorHandling(userId) {  
  try {  
    console.log(`Attempting to fetch user ${userId}...`);  
    const user = await fetchUserData(userId);  
    console.log('Success:', user);  
    return user;  
  } catch (error) {  
    console.error('Failed to fetch user:', error.message);  
    return null;  
  }  
}  
  
// Test with valid ID
```

```
getUserWithErrorHandling(105);

// Test with invalid ID (will trigger error)
setTimeout(() => {
  getUserWithErrorHandling(-1);
}, 1500);
```

Output:

```
Attempting to fetch user 105...
(after 1 second)
Success: { id: 105, name: 'David', email: 'david@example.com' }
(after 1.5 seconds)
Attempting to fetch user -1...
Failed to fetch user: Invalid user ID
```

Sequential vs Parallel Async Operations

```
// Sequential execution (operations wait for each other)
async function sequentialExecution() {
  console.log('Sequential execution started');
  const start = Date.now();

  const user = await fetchUser();          // Waits 500ms
  const posts = await fetchPosts();        // Waits 700ms
  const comments = await fetchComments(); // Waits 300ms

  const duration = Date.now() - start;
  console.log('Sequential total time:', duration + 'ms');
  console.log('Data:', { user, posts, comments });
}

// Parallel execution (operations run simultaneously)
async function parallelExecution() {
  console.log('\nParallel execution started');
  const start = Date.now();

  // Start all operations at once
  const [user, posts, comments] = await Promise.all([
    fetchUser(),
```

```
        fetchPosts(),
        fetchComments()
    ]);

    const duration = Date.now() - start;
    console.log('Parallel total time:', duration + 'ms');
    console.log('Data:', { user, posts, comments });
}

// Run both approaches
async function compareExecutions() {
    await sequentialExecution();
    await parallelExecution();
}

compareExecutions();
```

Output:

```
Sequential execution started
Sequential total time: 1500ms
Data: { user: { name: 'Alice' }, posts: ['Post1', 'Post2'], comments: ['Comment1', 'Comment2'] }

Parallel execution started
Parallel total time: 700ms
Data: { user: { name: 'Alice' }, posts: ['Post1', 'Post2'], comments: ['Comment1', 'Comment2'] }
```

Important Note: Parallel execution (700ms) is faster than sequential (1500ms) because operations run simultaneously rather than waiting for each to complete.

Real-World Example: Multiple Dependent Operations

```
// Realistic async/await pattern
function getUserById(userId) {
    return new Promise(resolve => {
        setTimeout(() => {
            resolve({ id: userId, name: 'Emma', role: 'admin' });
        }, 500);
    });
}
```

```
function getPermissions(role) {
  return new Promise(resolve => {
    setTimeout(() => {
      const permissions = {
        admin: ['read', 'write', 'delete'],
        user: ['read']
      };
      resolve(permissions[role] || []);
    }, 300);
  });
}

function logActivity(userId, action) {
  return new Promise(resolve => {
    setTimeout(() => {
      console.log(`Logged: User ${userId} performed ${action}`);
      resolve(true);
    }, 200);
  });
}

// Using async/await for clean sequential flow
async function authenticateUser(userId) {
  try {
    // Step 1: Get user
    console.log('Step 1: Fetching user...');
    const user = await getUserById(userId);
    console.log('User found:', user.name);

    // Step 2: Get permissions based on role
    console.log('\nStep 2: Fetching permissions...');
    const permissions = await getPermissions(user.role);
    console.log('Permissions:', permissions);

    // Step 3: Log the activity
    console.log('\nStep 3: Logging activity...');
    await logActivity(user.id, 'login');

    console.log('\nAuthentication complete!');
    return { user, permissions };
  } catch (error) {
    console.error('Authentication failed:', error.message);
    throw error;
  }
}
```

```
}  
  
authenticateUser(106);
```

Output:

```
Step 1: Fetching user...  
User found: Emma  
  
Step 2: Fetching permissions...  
Permissions: ['read', 'write', 'delete']  
  
Step 3: Logging activity...  
Logged: User 106 performed login  
  
Authentication complete!
```

4. EventEmitter: Custom Event-Driven Programming

What is EventEmitter?

EventEmitter is a core Node.js module that enables event-driven architecture. It allows objects to emit named events and register listener functions that execute when those events occur. This pattern is fundamental to Node.js and is used extensively in built-in modules like HTTP servers, streams, and file system watchers.

Think of EventEmitter like a notification system: one part of your application can broadcast events (like a radio station), and other parts can listen and react to those events (like radio receivers).

Basic EventEmitter Usage

```
// Importing EventEmitter (in Node.js)  
const EventEmitter = require('events');  
  
// Creating an emitter instance
```



```
const myEmitter = new EventEmitter();

// Registering an event listener
myEmitter.on('greet', () => {
  console.log('Hello! Event received.');
```

```
});

// Emitting the event
console.log('Emitting greet event...');
myEmitter.emit('greet');
console.log('Event emitted');
```

Output:

```
Emitting greet event...
Hello! Event received.
Event emitted
```

Passing Data with Events

```
const EventEmitter = require('events');
const emitter = new EventEmitter();

// Listener with parameters
emitter.on('userLoggedIn', (username, timestamp) => {
  console.log(`User ${username} logged in at ${timestamp}`);
});

// Emit event with data
emitter.emit('userLoggedIn', 'Alice', new Date().toISOString());
emitter.emit('userLoggedIn', 'Bob', new Date().toISOString());
```

Output:

```
User Alice logged in at 2026-02-16T10:30:00.000Z
User Bob logged in at 2026-02-16T10:30:00.000Z
```

Multiple Listeners for Same Event

```
const EventEmitter = require('events');
const orderEmitter = new EventEmitter();

// Multiple listeners for 'orderPlaced' event
orderEmitter.on('orderPlaced', (orderId) => {
  console.log(`[Email Service] Sending confirmation for order ${orderId}`);
});

orderEmitter.on('orderPlaced', (orderId) => {
  console.log(`[Inventory Service] Updating stock for order ${orderId}`);
});

orderEmitter.on('orderPlaced', (orderId) => {
  console.log(`[Notification Service] Pushing notification for order ${orderId}`);
});

// Emit event - all listeners execute
console.log('Processing order...');
orderEmitter.emit('orderPlaced', 'ORD-12345');
```

Output:

```
Processing order...
[Email Service] Sending confirmation for order ORD-12345
[Inventory Service] Updating stock for order ORD-12345
[Notification Service] Pushing notification for order ORD-12345
```

Creating Custom EventEmitter Classes

```
const EventEmitter = require('events');

// Custom class extending EventEmitter
class OrderProcessor extends EventEmitter {
  processOrder(orderId, items) {
    console.log(`Processing order ${orderId}...`);

    // Emit event when processing starts
    this.emit('orderStarted', orderId);
  }
}
```

```
// Simulate processing
setTimeout(() => {
  const total = items.reduce((sum, item) => sum + item.price, 0);

  // Emit event when processing completes
  this.emit('orderCompleted', {
    orderId,
    itemCount: items.length,
    total
  });
}, 1000);
}

cancelOrder(orderId) {
  console.log(`Cancelling order ${orderId}...`);
  this.emit('orderCancelled', orderId);
}
}

// Using the custom EventEmitter
const processor = new OrderProcessor();

// Register listeners
processor.on('orderStarted', (orderId) => {
  console.log(`✓ Order ${orderId} processing started`);
});

processor.on('orderCompleted', (orderData) => {
  console.log(`✓ Order ${orderData.orderId} completed`);
  console.log(`  Items: ${orderData.itemCount}, Total: $${orderData.total}`);
});

processor.on('orderCancelled', (orderId) => {
  console.log(`✗ Order ${orderId} has been cancelled`);
});

// Process an order
const items = [
  { name: 'Laptop', price: 999 },
  { name: 'Mouse', price: 25 }
];

processor.processOrder('ORD-001', items);

// Cancel an order after 2 seconds
```

```
setTimeout(() => {  
  processor.cancelOrder('ORD-002');  
}, 2000);
```

Output:

```
Processing order ORD-001...  
✓ Order ORD-001 processing started  
(after 1 second)  
✓ Order ORD-001 completed  
  Items: 2, Total: $1024  
(after 2 seconds)  
Cancelling order ORD-002...  
✗ Order ORD-002 has been cancelled
```

One-Time Event Listeners with once()

```
const EventEmitter = require('events');  
const gameEmitter = new EventEmitter();  
  
// Regular listener (fires every time)  
gameEmitter.on('playerScored', (player) => {  
  console.log(`${player} scored! (regular listener)`);  
});  
  
// One-time listener (fires only once)  
gameEmitter.once('playerScored', (player) => {  
  console.log(`${player} scored! (once listener - first goal)`);  
});  
  
// Emit event multiple times  
console.log('Goal 1:');  
gameEmitter.emit('playerScored', 'Alice');  
  
console.log('\nGoal 2:');  
gameEmitter.emit('playerScored', 'Bob');
```

```
console.log('\nGoal 3:');  
gameEmitter.emit('playerScored', 'Charlie');
```

Output:

```
Goal 1:  
Alice scored! (regular listener)  
Alice scored! (once listener - first goal)  
  
Goal 2:  
Bob scored! (regular listener)  
  
Goal 3:  
Charlie scored! (regular listener)
```

Notice: The `once()` listener only executes for the first event emission.

Removing Event Listeners

```
const EventEmitter = require('events');  
const notifier = new EventEmitter();  
  
// Define listener functions  
function alertUser(message) {  
  console.log(`ALERT: ${message}`);  
}  
  
function logMessage(message) {  
  console.log(`LOG: ${message}`);  
}  
  
// Register listeners  
notifier.on('notification', alertUser);  
notifier.on('notification', logMessage);  
  
console.log('With both listeners:');  
notifier.emit('notification', 'System update available');  
  
// Remove one listener
```

```
notifier.off('notification', alertUser);

console.log('\nAfter removing alert listener:');
notifier.emit('notification', 'New message received');

// Remove all listeners for an event
notifier.removeAllListeners('notification');

console.log('\nAfter removing all listeners:');
notifier.emit('notification', 'This will not be logged');
console.log('Event emitted but no listeners to handle it');
```

Output:

```
With both listeners:
ALERT: System update available
LOG: System update available

After removing alert listener:
LOG: New message received

After removing all listeners:
Event emitted but no listeners to handle it
```

Real-World Example: File Upload Progress Tracker

```
const EventEmitter = require('events');

class FileUploader extends EventEmitter {
  uploadFile(filename, sizeInMB) {
    console.log(`Starting upload: ${filename} (${sizeInMB}MB)`);
    this.emit('uploadStarted', filename);

    let uploaded = 0;
    const chunkSize = 10; // 10% chunks

    const interval = setInterval(() => {
      uploaded += chunkSize;

      // Emit progress event
```

```
this.emit('uploadProgress', {
  filename,
  progress: uploaded,
  total: 100
});

// Check if upload complete
if (uploaded >= 100) {
  clearInterval(interval);
  this.emit('uploadCompleted', {
    filename,
    size: sizeInMB,
    timestamp: new Date().toISOString()
  });
}
}, 300); // Simulate progress every 300ms
}
}

// Using the file uploader
const uploader = new FileUploader();

uploader.on('uploadStarted', (filename) => {
  console.log(`📁 Upload started: ${filename}`);
});

uploader.on('uploadProgress', (data) => {
  const bars = '█'.repeat(data.progress / 10);
  const spaces = '░'.repeat((data.total - data.progress) / 10);
  console.log(`Progress: [${bars}${spaces}] ${data.progress}%`);
});

uploader.on('uploadCompleted', (data) => {
  console.log(`✅ Upload completed: ${data.filename}`);
  console.log(`  Size: ${data.size}MB, Time: ${data.timestamp}`);
});

// Start upload
uploader.uploadFile('presentation.pdf', 25);
```

Output:

Starting upload: presentation.pdf (25MB)

📁 Upload started: presentation.pdf

Progress: [██████████] 10%

Progress: [██████████] 20%

Progress: [██████████] 30%

Progress: [██████████] 40%

Progress: [██████████] 50%

Progress: [██████████] 60%

Progress: [██████████] 70%

Progress: [██████████] 80%

Progress: [██████████] 90%

Progress: [██████████] 100%

☒ Upload completed: presentation.pdf

Size: 25MB, Time: 2026-02-16T10:30:03.000Z

Combining Patterns: A Complete Example

In real-world applications, you often combine multiple asynchronous patterns. Here's an example that uses EventEmitter with async/await:

```
const EventEmitter = require('events');

class DataPipeline extends EventEmitter {
  async processData(dataId) {
    try {
      this.emit('started', dataId);

      // Step 1: Fetch data
      this.emit('step', 'Fetching data...');
      await this.delay(500);
      const data = { id: dataId, raw: 'raw_data_content' };

      // Step 2: Validate data
      this.emit('step', 'Validating data...');
      await this.delay(300);
      if (!data.raw) {
        throw new Error('Invalid data');
      }

      // Step 3: Transform data
      this.emit('step', 'Transforming data...');
```



```
await this.delay(400);
const transformed = data.raw.toUpperCase();

// Step 4: Save data
this.emit('step', 'Saving data...');
await this.delay(300);

this.emit('completed', { dataId, result: transformed });

} catch (error) {
  this.emit('error', { dataId, error: error.message });
}
}

delay(ms) {
  return new Promise(resolve => setTimeout(resolve, ms));
}
}

// Using the pipeline
const pipeline = new DataPipeline();

pipeline.on('started', (dataId) => {
  console.log(`🌀 Pipeline started for data: ${dataId}`);
});

pipeline.on('step', (message) => {
  console.log(`⌚ ${message}`);
});

pipeline.on('completed', (result) => {
  console.log(`✅ Pipeline completed for data: ${result.dataId}`);
  console.log(`Result: ${result.result}`);
});

pipeline.on('error', (errorData) => {
  console.log(`❌ Pipeline failed for data: ${errorData.dataId}`);
  console.log(`Error: ${errorData.error}`);
});

// Execute
console.log('Starting data processing...\n');
pipeline.processData('DATA-001');
```

Output:

Starting data processing...

🔗 Pipeline started for data: DATA-001

⌚ Fetching data...

⌚ Validating data...

⌚ Transforming data...

⌚ Saving data...

✅ Pipeline completed for data: DATA-001

Result: RAW_DATA_CONTENT

Comparison: When to Use Which Pattern

Pattern	Best Used For	Advantages	Disadvantages
Callbacks	Simple async operations, legacy code, Node.js core APIs	• Simple concept • Universally supported • Low overhead	• Callback hell • Error handling repetition • Hard to maintain
Promises	Chaining async operations, multiple parallel operations	• Flat code structure • Better error handling • Native browser support • Promise.all/race utilities	• Still somewhat verbose • Need to understand chaining
Async/Await	Modern applications, sequential async operations, readable code	• Looks like sync code • Easy to read/write • Try-catch error handling • Best maintainability	• Requires understanding promises • Easy to accidentally

			write sequential code when parallel would be better
EventEmitter	Event-driven architectures, pub-sub patterns, real-time applications	• Decoupled components • Multiple listeners • Real-time reactions • Flexible architecture	• Can be complex • Memory leaks if listeners not removed • Harder to debug

Best Practices

1. Always Handle Errors

```
// ✗ Bad: Unhandled promise rejection
fetchData().then(data => console.log(data));

// ✓ Good: Always catch errors
fetchData()
  .then(data => console.log(data))
  .catch(error => console.error(error));

// ✓ Good: With async/await
async function getData() {
  try {
    const data = await fetchData();
    console.log(data);
  } catch (error) {
    console.error(error);
  }
}
```

2. Avoid Mixing Patterns Unnecessarily

```
// ✗ Bad: Mixing callbacks and promises
async function mixedApproach() {
  const data = await fetchData();
```

```
    processData(data, (error, result) => { // Callback in async function
      console.log(result);
    });
  }

// ☒ Good: Consistent pattern
async function consistentApproach() {
  const data = await fetchData();
  const result = await processDataPromise(data);
  console.log(result);
}
```

3. Remove Event Listeners to Prevent Memory Leaks

```
// ☒ Good practice
const emitter = new EventEmitter();

function handler(data) {
  console.log(data);
}

emitter.on('event', handler);

// When done, remove listener
emitter.off('event', handler);

// Or use once() for one-time events
emitter.once('event', handler);
```

4. Use Parallel Execution When Possible

```
// ☒ Slow: Sequential when operations are independent
async function sequential() {
  const user = await fetchUser();      // 500ms
  const posts = await fetchPosts();    // 500ms
  const comments = await fetchComments(); // 500ms
  // Total: 1500ms
}

// ☒ Fast: Parallel execution
async function parallel() {
  const [user, posts, comments] = await Promise.all([
```

```
    fetchUser(),
    fetchPosts(),
    fetchComments()
  ]);
  // Total: 500ms (time of slowest operation)
}
```

Common Pitfalls and Solutions

Pitfall 1: Forgetting to Return Promises in Chains

```
// ✗ Wrong: Not returning promise
getUserPromise(1)
  .then(user => {
    getUserOrdersPromise(user.id); // Missing return!
  })
  .then(orders => {
    console.log(orders); // undefined!
  });

// ✓ Correct: Return the promise
getUserPromise(1)
  .then(user => {
    return getUserOrdersPromise(user.id);
  })
  .then(orders => {
    console.log(orders); // Correct data
  });
```

Pitfall 2: Not Awaiting Async Functions

```
// ✗ Wrong: Forgetting await
async function getData() {
  const data = fetchData(); // Returns promise, not data!
  console.log(data); // Logs Promise object
}

// ✓ Correct: Use await
async function getData() {
```

```
const data = await fetchData();
console.log(data); // Logs actual data
}
```

Pitfall 3: Too Many Event Listeners

```
// ✗ Potential memory leak
for (let i = 0; i < 1000; i++) {
  emitter.on('event', () => console.log(i));
}
// Now you have 1000 listeners!

// ☑ Better: Use one listener
emitter.on('event', (data) => {
  // Handle all cases in one listener
});
```

Interview Questions

Theoretical Questions

1. What is the error-first callback convention in Node.js? Why is it important? 

Answer: The error-first callback convention places the error parameter as the first argument of callback functions. The first parameter is null or undefined if no error occurred, otherwise it contains an error object. This convention provides consistent error handling across Node.js APIs and makes it clear that errors must be handled before processing results.

2. Explain the difference between `Promise.all()` and `Promise.race()`.

Answer: `Promise.all()` waits for all promises to resolve and returns an array of all results. If any promise rejects, `Promise.all()` immediately rejects. `Promise.race()` returns the result of the first promise to settle (resolve or reject), ignoring all other promises. `Promise.all()` is used when you need all results; `Promise.race()` is used when you need the fastest response.

3. What happens if you don't use try-catch with async/await?

Answer: Without try-catch, any rejected promise will result in an unhandled promise rejection. In Node.js, this will log a warning and in future versions may terminate the process. Unhandled rejections make debugging difficult as errors are not caught where they occur. Always use try-catch or attach .catch() to the async function call.

4. How does EventEmitter differ from Promises?

Answer: EventEmitter is for pub-sub patterns where multiple listeners can react to events that may occur multiple times. Promises represent a single asynchronous operation that resolves once. EventEmitters are suitable for ongoing events (like server requests), while Promises are for one-time operations (like fetching data). EventEmitters allow multiple handlers; Promises have single resolution.

5. What is the purpose of the finally() method in promises?

Answer: finally() executes code regardless of whether the promise resolves or rejects. It's used for cleanup operations like closing connections, hiding loading spinners, or releasing resources. The callback receives no arguments because it runs in both success and failure cases. finally() returns a promise, allowing further chaining.

6. Can you use await outside of an async function? If not, how do you handle top-level awaits?

Answer: In traditional JavaScript, await can only be used inside async functions. However, ES2022 introduced top-level await for modules (when using type: "module"). Before that, you'd wrap code in an async IIFE: (async () => { await someFunction(); })(); or use .then() at the top level.

7. What are the three states of a Promise?

Answer: The three states are: (1) Pending - initial state, neither fulfilled nor rejected, (2) Fulfilled - operation completed successfully, the promise has a value, (3) Rejected - operation failed, the promise has a reason (error). A promise can transition from pending to fulfilled or rejected, but once settled, its state cannot change.

8. Explain callback hell and how Promises solve it.

Answer: Callback hell occurs when multiple asynchronous operations are nested, creating deeply indented, hard-to-read code. Each callback is nested inside the previous one, forming a "pyramid of doom". Promises solve this through chaining with .then(), allowing flat, sequential code. Each .then() returns a new promise, enabling the next operation to chain naturally without nesting.

9. When would you use EventEmitter.once() instead of EventEmitter.on()?

Answer: Use `once()` when you want a listener to execute only for the first occurrence of an event, then automatically remove itself. It's ideal for initialization events, first-time setup, or events that should trigger actions only once (like 'ready' or 'connected' events). Use `on()` when the listener should handle every occurrence of the event.

10. What is the difference between sequential and parallel async operations? When would you use each?

Answer: Sequential operations wait for each operation to complete before starting the next (await one after another). Parallel operations start simultaneously (Promise.all). Use sequential when operations depend on each other (need previous result), or when rate limiting is required. Use parallel when operations are independent to improve performance by reducing total wait time.

Practical Questions for Students to Solve

1. Question 1: Convert Callback to Promise

Convert the following callback-based function to return a Promise:

```
function readFileCallback(filename, callback) {
  setTimeout(() => {
    if (filename === 'data.txt') {
      callback(null, 'File content here');
    } else {
      callback(new Error('File not found'), null);
    }
  }, 1000);
}
```

2. Question 2: Implement Retry Logic

Write an async function that retries a failed operation up to 3 times before giving up. The function should:

- Accept a promise-returning function as a parameter
- Retry on failure with 1-second delay between attempts
- Return the result on success or throw error after 3 failures

3. Question 3: Build a Simple Task Queue with EventEmitter

Create a TaskQueue class that:

- Extends EventEmitter
- Accepts tasks and processes them one at a time
- Emits 'taskStarted', 'taskCompleted', and 'queueEmpty' events
- Each task is an async function that takes time to complete

4. Question 4: Parallel Data Processing

You have an array of user IDs. Write an async function that:

- Fetches user data for all IDs in parallel
- Filters out users with errors (don't let one failure break all)
- Returns only successfully fetched users
- Hint: Use Promise.allSettled()

5. Question 5: Implement a Simple Promise

Create your own MyPromise class that mimics basic Promise behavior with:

- Constructor accepting an executor function
- then() method for success handling
- catch() method for error handling
- Proper state management (pending, fulfilled, rejected)

Summary

Asynchronous patterns are the backbone of Node.js applications. Understanding the evolution from callbacks to promises to async/await helps you write more maintainable code. EventEmitter adds another dimension for event-driven architectures.

Key Takeaways:

- Callbacks are simple but lead to nested code and repetitive error handling
- Promises provide chainable operations with cleaner error handling
- Async/await makes asynchronous code look synchronous, improving readability
- EventEmitter enables decoupled, event-driven communication between components



- Choose the right pattern based on your use case: promises for single operations, `async/await` for readability, `EventEmitter` for pub-sub patterns
- Always handle errors appropriately with each pattern
- Use parallel execution (`Promise.all`) when operations are independent
- Remember to clean up event listeners to prevent memory leaks

Mastering these patterns will enable you to build efficient, scalable Node.js applications and handle the asynchronous nature of JavaScript with confidence.

